DUOC UC · SEDE PUERTO MONTT
Escuela de Informática y Telecomunicaciones

Informe de avance funcional del proyecto VcM

Ofta_App

Evaluación Parcial 2

Asignatura: Desarrollo de Aplicaciones Móviles (DSY1105)

Equipo: Sin Economía Grupo N.º 7

Integrantes: Diego Andrés Díaz Hernández

Martín Andrés Díaz González

Docente: Marcelo Eduardo Crisóstomo Carrasco

Fecha: Octubre de 2026

1. Introducción

Este informe presenta el estado actual de Ofta_App respecto de la Evaluación Parcial 2 de Desarrollo de Aplicaciones Móviles. La evaluación revisa el avance real de la aplicación hasta la clase 11 y no exige que el proyecto esté terminado. Por eso, el objetivo es identificar qué ya funciona, qué está a medias y qué falta para cumplir la pauta antes de la entrega.

La revisión se hizo leyendo el código de la aplicación, el repositorio de GitHub y los documentos del curso (pauta de evaluación, caso OftApp y transcripción de la reunión con el cliente). **No se compiló ni se ejecutó la aplicación**, así que ninguna función está probada en el teléfono o el emulador. Tampoco se pudo revisar Trello. El código analizado viene de una copia descargada del repositorio, que podría no coincidir con el último commit. Cuando decimos que algo está implementado, significa que existe en el código y parece correcto, pero todavía debe comprobarse en ejecución.

2. Descripción del proyecto

Ofta_App es una aplicación Android para registrar y consultar exámenes oftalmológicos con datos ficticios. Está desarrollada en Kotlin con Jetpack Compose y Material 3, y sigue la arquitectura MVVM.

El caso lo plantea MG Ingeniería Informática SPA a partir de un cliente que realiza exámenes oftalmológicos con equipos que entregan los resultados en PDF, Excel o imágenes. Esos archivos se guardan en carpetas locales, se envían por correo o se entregan impresos, y no existe una ficha digital única por paciente. Esto dificulta encontrar los resultados y revisar los antecedentes de cada persona.

En la reunión del 15 de septiembre de 2026, el representante del cliente explicó que se busca reunir los exámenes en una sola aplicación móvil. La idea es que el tecnólogo médico pueda cargarlos, que el médico revise el historial del paciente y que el paciente pueda consultarlos y llevarlos consigo si quiere una segunda opinión. También pidió trabajar con datos ficticios y considerar la accesibilidad visual, porque los usuarios son pacientes oftalmológicos.

3. Avances realizados

Según el código, la aplicación cuenta hoy con las siguientes funciones:

- **Inicio de sesión y roles.** Hay cuatro roles (paciente, TENS, oftalmólogo y administrador) y cuentas de prueba para tres de ellos. Sirve para separar a quien carga exámenes de quien solo consulta.
- **Menú principal.** Combina un menú lateral con tarjetas de acceso rápido y un resumen con el total de exámenes, completados, pendientes y pacientes.
- **Listado y búsqueda de exámenes.** Muestra los exámenes guardados y permite buscar por tipo de examen o por nombre del paciente. También hay listas por estado desde el resumen.
- **Detalle del examen.** Muestra paciente, tipo, ojo evaluado, estado, observaciones, fecha, profesional y sucursal.

- **Historial y pacientes.** El historial agrupa los exámenes por paciente, lo que ayuda a ver los antecedentes en un solo lugar. Además hay un listado de pacientes.
- **Registro de examen.** Es un formulario para ingresar un nuevo examen, disponible solo para roles que no sean paciente.
- **PDF del examen.** Desde el detalle se genera un PDF con los datos del examen y se abre con una aplicación del teléfono.
- **Notificación y correo de prueba.** Existe un botón de prueba que muestra una notificación y abre el correo.

Los datos se leen y se guardan en una base de datos remota (PostgreSQL en Neon). La aplicación todavía no usa Room.

4. Evaluación de los requisitos

La tabla resume los siete indicadores oficiales de la Evaluación Parcial 2, con su ponderación. Las ponderaciones indican cuánto vale cada indicador y no corresponden a una nota. Los estados se basan solo en la revisión del código.

Requisito (ponderación)	Estado actual	Qué se ha logrado y qué falta
Interfaz y navegación (15%)	**Parcial**	Hay pantallas de inicio de sesión, menú, listado, detalle, historial, registro y perfil, con Material 3 y colores propios, y se puede pasar de una a otra. Falta poder llegar al detalle desde el historial, conservar la pantalla al girar el teléfono y retirar elementos que no hacen nada, como el texto resultado.pdf del formulario y el botón de contraseña olvidada.
Formularios y validaciones (15%)	**Parcial**	El inicio de sesión y el registro de examen validan campos vacíos, fecha y credenciales, y muestran mensajes. Falta mostrar el error junto a cada campo, aceptar fechas como 1/1/2026 (hoy pasan la validación pero fallarían al guardar), confirmar solo cuando el examen realmente se guardó y poder adjuntar un documento.
Estado, ViewModel y lógica de validación (10%)	**Parcial**	Hay tres ViewModel con StateFlow (sesión, menú y exámenes) y las pantallas se actualizan con ellos. Falta mover la validación del formulario al ViewModel, mostrar estados de carga y error, y evitar que las consultas a la base de datos congelen la pantalla.
Listas, búsquedas, filtros y detalles (10%)	**Parcial**	La lista se llena con datos reales, tiene búsqueda de texto, mensaje cuando no hay resultados, detalle e historial por paciente. Faltan filtros por estado, fecha y sucursal, y búsqueda en el historial y en pacientes.
Arquitectura MVVM y persistencia Room (15%)	**Parcial** (Room pendiente)	El código está separado en model, repository, viewmodel y ui. Room no está implementado: la información se guarda en una base remota, por lo que la app no funciona sin Internet. Además, cada ViewModel crea su propio repositorio.

Requisito (ponderación)	Estado actual	Qué se ha logrado y qué falta
GitHub, documentación y planificación (20%)	**Parcial** (Trello no verificado)	El repositorio tiene 27 commits entre el 12 de septiembre y el 13 de octubre, de dos cuentas, además de README y las evidencias de las clases 1 y 2. Falta equilibrar los aportes, ya que una cuenta tiene 23 commits y la otra 4. Además, 18 commits son del 13 de octubre y el README está desactualizado. No se pudo revisar la planificación en Trello.
Recursos del dispositivo Android (15%)	**Parcial**	La generación del PDF y su apertura con otra aplicación está implementada. La notificación no se mostraría en Android 13 o superior porque nunca se pide el permiso, y el correo es solo una prueba. Falta un segundo recurso útil para el flujo, como un selector de documentos, un selector de fecha o el estado de la conexión.

Con esta revisión ningún indicador se puede dar por cumplido por completo. Los que más pesan hoy son la falta de Room, la falta de adjuntar documentos y la evidencia de trabajo en equipo.

5. Principales aspectos por mejorar

Persistencia con Room. Es un requisito explícito de la evaluación y todavía no existe. Hoy la información depende de una base remota, por lo que sin Internet no se puede ver ni guardar nada. Hay que crear la base local con Room y hacer que la aplicación la use para guardar y consultar los exámenes.

Seguridad de la base de datos. La aplicación se conecta a Neon con el usuario y la contraseña escritos en el código. Algunos commits intentaron reducir esta exposición, pero la clave sigue en el código y en el historial de GitHub. Hay que cambiar la contraseña y sacar las claves del proyecto. El caso también pide no usar credenciales ni bases de producción.

Adjuntar y ver documentos. El problema central del cliente es reunir los resultados de los exámenes, pero hoy el formulario solo muestra un texto fijo resultado.pdf y no permite elegir un archivo. Falta adjuntar un PDF o una imagen al examen y poder verlo desde el detalle.

Formulario de registro. Además de lo indicado en la tabla, la lista de profesionales está escrita en el código y algunos nombres no coinciden con los de la base de datos, por lo que el examen podría quedar sin profesional asignado. Si no hay pacientes, la opción Sin pacientes se puede elegir y crearía un paciente con ese nombre.

Búsquedas y filtros. La tarjeta de inicio dice que se puede buscar por fecha, tipo o estado, pero el buscador solo funciona con texto. Además, cada letra escrita consulta de nuevo toda la base de datos.

Permisos y recursos del dispositivo. Hay que pedir el permiso de notificaciones cuando se use, o retirar la notificación y el correo de prueba. Conviene que los dos recursos finales tengan sentido para el flujo, por ejemplo elegir el documento del examen y elegir la fecha con un calendario.

Acceso según el rol. El paciente puede ver todos los pacientes y exámenes, no solo los suyos. También falta que el personal autorizado pueda actualizar un examen o cambiar su estado, como pide el caso.

Conexión, errores y estabilidad. Las consultas a la base de datos pueden dejar la pantalla congelada y no hay manejo de errores. Si no hay Internet o la base no responde, la aplicación podría cerrarse. Esto no se ha probado y debe comprobarse en un emulador, con y sin conexión.

README, GitHub y planificación. El README dice que los datos están en memoria y que no se necesita Room, lo que no coincide con el código. Tampoco incluye integrantes ni instrucciones para ejecutar el proyecto. Los dos integrantes deberían registrar commits hasta la entrega, y hay que mostrar el tablero de Trello con tareas y responsables.

6. Plan de trabajo

Las tareas están ordenadas desde las más importantes hasta las mejoras secundarias.

Tarea pendiente	Prioridad	Cómo se comprobará que funciona
Compilar la app y probarla en un emulador, con y sin Internet	Alta	La app se instala, se inicia sesión y se recorren las pantallas sin cierres inesperados.
Implementar Room y usarlo para guardar y consultar exámenes y pacientes	Alta	Se registra un examen, se cierra la app por completo y al abrirla el examen sigue en la lista, incluso sin Internet.
Cambiar la contraseña de la base remota y sacar las claves del código	Alta	No quedan claves ni direcciones de conexión en el proyecto y la app sigue funcionando.
Corregir el formulario de registro (fecha con calendario, errores por campo, confirmación solo si se guardó, quitar la opción Sin pacientes)	Alta	Se prueba con campos vacíos, fecha inválida, fecha como 1/1/2026 y datos correctos, sin cierres y con mensajes claros.
Adjuntar un PDF o imagen al examen y verlo en el detalle	Alta	Se elige un archivo, se guarda, se vuelve a abrir el examen y el archivo se puede ver.
Dejar dos recursos del dispositivo funcionando, con sus permisos	Alta	Cada recurso se ejecuta y muestra su resultado; el permiso se pide cuando corresponde.
Mostrar commits de ambos integrantes, actualizar el README y presentar Trello	Alta	El historial muestra aportes de los dos y el README permite abrir y ejecutar el proyecto desde cero.
Agregar filtros por estado, sucursal y fecha, y búsqueda en historial y pacientes	Media	Se combinan texto y filtros, y la lista se actualiza después de registrar un examen.
Limitar lo que ve cada rol y permitir cambiar el estado de un examen	Media	Con cada usuario de prueba se verifica qué datos y qué acciones tiene disponibles.

Tarea pendiente	Prioridad	Cómo se comprobará que funciona
Mostrar estados de carga y error, y crear pruebas unitarias de las validaciones	Media	Sin Internet aparece un mensaje en vez de un cierre, y las pruebas se ejecutan sin fallas.
Mejorar accesibilidad y detalles de diseño, y reemplazar datos de ejemplo que parezcan reales	Baja	Se revisa con textos más grandes y con TalkBack; no quedan correos ni datos personales reales.

7. Conclusión

Ofta_App tiene una base sólida para esta etapa: una interfaz ordenada con Material 3, inicio de sesión por roles, listado con búsqueda, detalle, historial, formulario de registro y generación de PDF, todo organizado en las capas de MVVM. Estas funciones responden a una parte importante del caso, que es reunir y consultar exámenes oftalmológicos en un solo lugar.

Sin embargo, el proyecto todavía no cumple varios requisitos obligatorios. Falta Room, no se pueden adjuntar los documentos de los exámenes, solo hay un recurso del dispositivo claramente funcional y las claves de la base remota están expuestas. Además, no se ha comprobado la ejecución de la app y falta evidencia de planificación en Trello. Lo primero será compilar y probar la aplicación, implementar Room y corregir el registro de exámenes con el adjunto de documentos, mientras el equipo ordena el README y el historial de GitHub.

Anexo de evidencias

Archivos del proyecto que respaldan los hallazgos principales (ruta base: app/src/main/java/com/equipo7/oftaapp).

Hallazgo	Archivo
Conexión a la base remota, claves y consultas	repository/ExamenRepository.kt
Estado y carga de datos	viewmodel/ExamenViewModel.kt, viewmodel/MenuViewModel.kt
Formulario de registro, validación de fecha y texto fijo resultado.pdf	ui/screens/RegistrarExamenScreen.kt
Navegación, roles y usuarios de prueba	ui/screens/MenuPrincipalScreen.kt, model/Usuario.kt, repository/UsuarioRepository.kt
PDF, notificación y correo de prueba	util/PdfReport.kt, notifications/NotificationHelper.kt, ui/screens/DetalleExamenScreen.kt
Dependencias (no incluyen Room)	app/build.gradle.kts, gradle/libs.versions.toml
Historial de commits y documentación	Registro de commits del repositorio, README.md

No se encontraron referencias a Room, cámara, galería ni selector de documentos en el proyecto. Los commits se revisaron con el registro entregado por el equipo.